

Instalación de Docker en Linux

Proyecto CodeByMike

Michael David Rodríguez Beltran

Instructora: Diana Delgado.

Servicio Nacional de Aprendizaje (SENA)

Centro de Servicios Financieros

Análisis y Desarrollo de Software, Ficha 3114731

Tabla de Contenido

Introducción.....	4
Por Qué Mi Instalación Se Ve Distinta a la de la Guía.....	5
Dejar Docker Funcionando en Ubuntu.....	6
Instalación.....	6
Los Dos Pasos que Sí Son Propios de Linux.....	7
Comprobar que el Motor Responde.....	8
Cada Pantalla de Docker Desktop y su Comando.....	9
Qué Aplicación Usé en Lugar de Dino Run.....	10
Los Seis Ejercicios.....	11
Ejercicio 1. Comprobar que el Motor Sirve.....	11
Ejercicio 2. Construir la Imagen y Revisarla.....	12
Ejercicio 3. Levantar Todo con Docker Compose.....	15
Ejercicio 4. Las Variables de Entorno y la Clave Secreta.....	17
Ejercicio 5. Que los Datos No Se Pierdan.....	19
Ejercicio 6. El Puerto Ocupado.....	21
Las Preguntas de la Sección 17.....	23
Qué Significa Empaquetar una Aplicación.....	23
Diferencia entre Imagen, Contenedor y Dockerfile.....	23
Sobre las Pantallas y las Capas.....	24
Qué Captura Corresponde a Cada Criterio.....	24
Conclusiones.....	25
Referencias.....	26

Resumen

Este documento recoge el taller de Docker, resuelto en un computador con Ubuntu 24 y no en Windows con Docker Desktop, que es el entorno que la guía explica.

Explico que pasos de la guía no aplican en Linux y por qué, y dejo una tabla que empareja cada pantalla de Docker Desktop con el comando que hace lo mismo en la terminal.

Los seis ejercicios de la página 27 los desarrollé sobre mi propio proyecto, CodeByMike, que ya usa Docker para levantar sus bases de datos de desarrollo, en lugar de la aplicación de ejemplo Dino Run. Los temas que evalúa la guía siguen siendo los mismos en este proyecto.

Introducción

Docker sirve para empaquetar una aplicación con todo lo que necesita para correr y ejecutarla como un proceso aislado, al que se le llama contenedor.

La guía del curso enseña todo esto en Windows, con Docker Desktop y WSL. Mi computador tiene Ubuntu 24 y ahí gran parte de esa instalación no se puede hacer. No porque sea difícil, sino porque el problema que esos pasos resuelven no existe en Linux.

Para no entregar un trabajo incompleto, lo que hice fue ir paso por paso de la guía y responder dos cosas: si el paso aplica en Linux, cómo se hace; y si no aplica, por qué no.

Por Qué Mi Instalación Se Ve Distinta a la de la Guía

Windows no tiene el núcleo de Linux, así que instala una máquina virtual pequeña con el núcleo real adentro, y sobre esa máquina Docker Desktop levanta el motor. Esto me parece importante en la adaptación, cuando alguien sigue la guía en Windows, la aplicación tampoco está corriendo en Windows. Está corriendo en un Linux escondido dentro de WSL. Docker Desktop es lo que hace que el usuario de windows no se de cuenta.

En mi caso el núcleo ya es el del sistema, así que no hay nada que instalar por debajo. No hay WSL, no hay que habilitar virtualización en la BIOS y no hay máquina virtual intermedia.

Docker queda como un servicio del sistema y se maneja desde la terminal. En la Tabla 1 dejo la comparación completa.

Tabla 1

Comparación entre lo que pide la guía y lo que hice en Linux

Paso de la guía	Qué pasa en mi equipo	Con qué lo reemplacé
Instalar Docker Desktop (sección 4)	En Linux, Docker Desktop es opcional. Solo agrega una interfaz gráfica y una máquina virtual que no necesito. Uso Docker Engine directo.	<code>docker --version</code> y <code>docker info</code> mostrando el motor
Instalar WSL 2 (sección 5.1)	No existe en Linux. El núcleo ya es el del sistema.	<code>uname -r</code> , que muestra el núcleo que comparten los contenedores
Habilitar virtualización en BIOS (sección 5.2)	No se necesita. Los contenedores no son máquinas virtuales, son procesos aislados.	<code>docker info</code> , donde aparecen <code>apparmor</code> , <code>seccomp</code> y <code>cgroupns</code>
Esperar el estado Engine running (sección 5.3)	El motor es un servicio de <code>systemd</code> , no una aplicación que uno abre.	<code>systemctl is-active docker</code> y <code>systemctl is-enabled docker</code>
Pantallas Builds, Images, Containers, Logs y Bind mounts	No tengo interfaz gráfica, pero cada pantalla tiene su comando.	Tabla 2, con la lista completa
Usar Gordon	Gordon viene con Docker Desktop, así que no lo tengo.	Los comandos que Gordon ejecuta por dentro. La misma guía los menciona
Archivo <code>iniciar_juego.bat</code>	Es un archivo de Windows.	Los scripts <code>npm run db:up</code> y <code>npm run db:seed</code> de mi proyecto

Todo lo que aparece en este trabajo, lo corrí en un pc con Ubuntu 24.04.4 LTS, núcleo 7.0.0-28-generic, Docker Engine 29.6.2 y Docker Compose v5.3.1.

Dejar Docker Funcionando en Ubuntu

Esta parte reemplaza las secciones 4, 5 y 15.1 de la guía.

Instalación

Ubuntu trae Docker en sus repositorios, pero casi siempre es una versión vieja y además no incluye Compose v2. Por eso se instala desde el repositorio oficial de Docker (Docker Inc., 2026c). Estos son los comandos:

```
# quitar los paquetes viejos que trae la distribución
sudo apt remove docker docker-engine docker.io containerd runc

# agregar la llave y el repositorio oficial
sudo apt update && sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# instalar el motor, el cliente y los complementos
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

Lo de la llave GPG no es puro trámite. Sirve para que apt verifique que los paquetes vienen firmados por Docker y no los cambió alguien en el camino.

Los Dos Pasos que Sí Son Propios de Linux

Después de instalar quedan dos ajustes. Estos son el equivalente de esperar a que Docker Desktop diga Running.

El primero es dejar el servicio andando y que arranque solo con el computador. Vale la pena separar las dos cosas, porque no son lo mismo: `active` quiere decir que está corriendo ahorita, y `enabled` quiere decir que vuelve a arrancar cuando reinicie el equipo. En las evidencias muestro las dos.

```
sudo systemctl enable --now docker
systemctl is-active docker      # -> active
systemctl is-enabled docker     # -> enabled
```

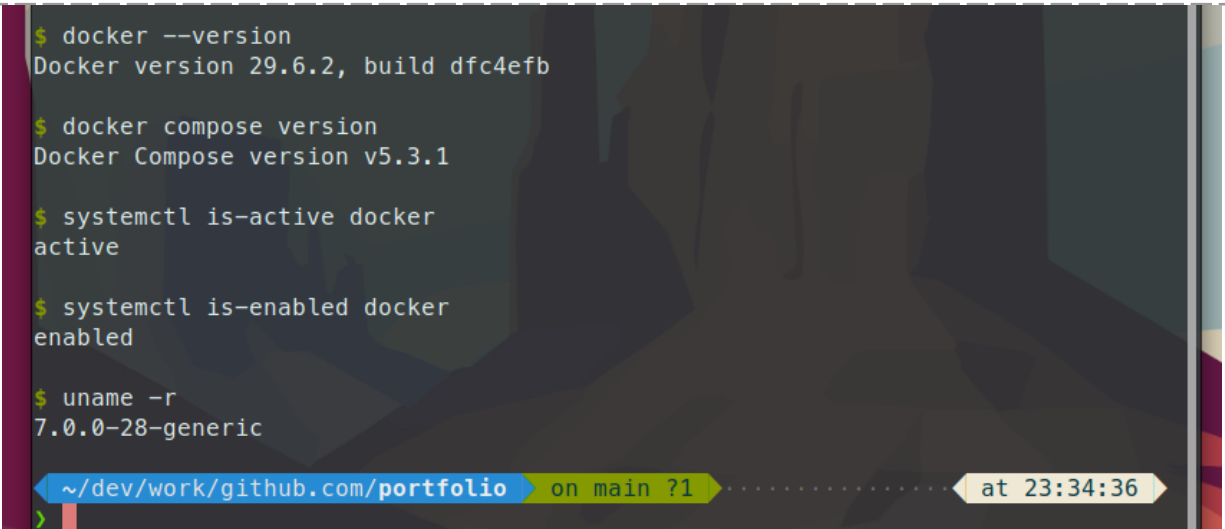
El segundo es poder usar Docker sin escribir `sudo` cada vez. El demonio atiende en un socket, `/var/run/docker.sock`, que pertenece al grupo `docker`. Si mi usuario no está en ese grupo, todos los comandos piden contraseña.

```
sudo usermod -aG docker $USER
newgrp docker                # o cerrar sesión y volver a entrar
id -nG | tr ' ' '\n' | grep -x docker    # -> docker
```

Sobre esto hay algo que aprendí leyendo la documentación y que me parece importante anotar: estar en el grupo `docker` es casi lo mismo que ser administrador del equipo. Quien pueda hablarle al socket puede montar todo el disco dentro de un contenedor y hacer lo que quiera. No es un error de configuración, es como está hecho Docker. Si uno necesita algo más cerrado existe el modo `rootless`, que corre el motor con el usuario normal (Docker Inc., 2026j). En Windows uno nunca ve este detalle porque Docker Desktop lo tapa.

Figura 1

Versiones instaladas y estado del servicio

A terminal window with a dark background and light-colored text. The terminal shows five commands and their outputs: 'docker --version' returns 'Docker version 29.6.2, build dfc4efb'; 'docker compose version' returns 'Docker Compose version v5.3.1'; 'systemctl is-active docker' returns 'active'; 'systemctl is-enabled docker' returns 'enabled'; and 'uname -r' returns '7.0.0-28-generic'. At the bottom, there is a status bar with a blue segment showing the path '~ / dev / work / github.com / portfolio', a green segment showing 'on main ?1', and a yellow segment showing 'at 23:34:36'.

```
$ docker --version
Docker version 29.6.2, build dfc4efb

$ docker compose version
Docker Compose version v5.3.1

$ systemctl is-active docker
active

$ systemctl is-enabled docker
enabled

$ uname -r
7.0.0-28-generic

~/dev/work/github.com/portfolio on main ?1 at 23:34:36
```

Donde: Terminal, en cualquier carpeta.

Comandos a ejecutar:

```
docker --version
docker compose version
systemctl is-active docker
systemctl is-enabled docker
uname -r
```

Debe verse: Las cinco salidas seguidas en la misma terminal. Esta captura reemplaza la del panel de Docker Desktop con el letrero Engine running.

Figura 2*Docker funcionando sin sudo*

```

$ id -nG | tr ' ' '\n' | grep -x docker
docker

$ docker ps --format 'table {{.Names}}\t{{.Status}}'
NAMES                                STATUS
codebymike-libsqli-demo              Up 2 days
codebymike-libsqli-main               Up 2 days
supabase_db_fiscaliso                 Up 2 days (healthy)
supabase_studio_fiscaliso             Up 2 days (healthy)
supabase_pg_meta_fiscaliso            Up 2 days (healthy)
supabase_storage_fiscaliso            Up 2 days (healthy)
supabase_rest_fiscaliso               Up 2 days
supabase_realtime_fiscaliso           Up 2 days (healthy)
supabase_inbucket_fiscaliso           Up 2 days (healthy)
supabase_auth_fiscaliso               Up 2 days (healthy)
supabase_kong_fiscaliso               Up 2 days (healthy)
supabase_vector_fiscaliso             Up 2 days (healthy)
supabase_analytics_fiscaliso          Up 2 days (healthy)

```

~/dev/work/github.com/portfolio on main ?1 at 23:35:53

Donde: Terminal, en cualquier carpeta.**Comandos a ejecutar:**

```

id -nG | tr ' ' '\n' | grep -x docker
docker ps

```

Debe verse: La palabra docker como respuesta del primer comando y enseguida la lista de contenedores, sin haber escrito sudo.*Nota.* Captura tomada en Ubuntu 24.04.4 LTS con Docker Engine 29.6.2. Elaboración propia.

Comprobar que el Motor Responde

Los cuatro comandos que pide la sección 5.3 de la guía funcionan igual en Linux, porque son del cliente y no de la interfaz gráfica:

```

docker --version
docker compose version
docker info
docker run hello-world

```

Aparte de esos revisé un par de cosas que solo tienen sentido en Linux:

```
uname -r
systemctl status docker --no-pager | head -12
docker info --format 'Motor: {{.ServerVersion}} | Datos: {{.DockerRootDir}}'
docker info --format 'Seguridad: {{.SecurityOptions}}'
```

Ahí me encontré un detalle curioso. Al correr `docker context ls` me aparecen dos contextos:

`default`, que es el que uso, y `desktop-linux`, que quedó de una prueba vieja de Docker Desktop.

Vale la pena mirarlo antes de empezar, porque si uno está en el contexto equivocado los

contenedores parecen desaparecer y en realidad están en el otro motor.

Figura 3

Salida de docker info

Donde:
Terminal,
en
cualquier
carpeta.

```
mike@mike-dev:~/dev/work/github.com/portfolio
$ docker info | head -32
Client: Docker Engine - Community
Version: 29.6.2
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.35.0
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v5.3.1
    Path: /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 18
  Running: 13
  Paused: 0
  Stopped: 5
Images: 33
Server Version: 29.6.2
Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog

$ docker context ls
NAME                DESCRIPTION          DOCKER ENDPOINT
default *           Current DOCKER_HOST based configuration   unix:///var/run/docker
.sock
desktop-linux       Docker Desktop              unix:///home/mike/.doc
ker/desktop/docker.sock

~/dev/work/github.com/portfolio on main ?1 at 23:37:11
```

Comandos a ejecutar:

```
docker info | head -40
docker context ls
```

Debe verse: El bloque Server con la versión del motor y la carpeta de datos, y en Security Options los valores apparmor, seccomp y cgroupns, que son los mecanismos de aislamiento del núcleo. Esta captura hace las veces de la de virtualización habilitada en BIOS que pide la sección 5.2.

Cada Pantalla de Docker Desktop y su Comando

Esta tabla es la que más me sirvió mientras hacía el taller, porque con ella pude seguir la guía sin saltarme nada. En varios casos el comando muestra más información que la pantalla.

Tabla 2

Equivalencia entre las pantallas de Docker Desktop y los comandos

Pantalla en Docker Desktop	Comando en Linux
Panel principal, Engine running	<code>systemctl is-active docker</code>
Images	<code>docker images</code>
Containers	<code>docker ps -a</code>
Builds	<code>docker buildx history ls</code>
Builds, pestaña Logs	<code>docker buildx history logs <ID></code>
Builds, pestaña Source	<code>docker buildx history inspect <ID></code>
Containers, pestaña Logs	<code>docker logs <nombre></code>
Containers, pestaña Inspect	<code>docker inspect <nombre></code>
Containers, pestaña Files	<code>docker exec -it <nombre> sh</code>
Containers, pestaña Stats	<code>docker stats</code>
Containers, pestaña Bind mounts	<code>docker inspect --format '{{json .Mounts}}' <nombre></code>
Volumes	<code>docker volume ls</code>
Capas de una imagen	<code>docker history <imagen></code>
Grupo de un proyecto de Compose	<code>docker compose ps</code>

Nota. Elaboración propia. Probé todos los comandos en Docker Engine 29.6.2.

Qué Aplicación Usé en Lugar de Dino Run

La guía usa Dino Run porque necesita alguna aplicación para meter en un contenedor. Yo aproveché mi proyecto de portafolio, CodeByMike, que ya venía usando Docker antes del curso. Eso me obligó a explicar algo que al principio pensé que era una falla del proyecto y resultó ser una decisión: mi aplicación no se despliega en contenedores. La publica Vercel. Si la metiera en un contenedor para producción perdería los despliegues de prueba por cada cambio y la reversión automática, y no ganaría nada.

Entonces, dónde uso Docker? En dos cosas, y las dos me sirvieron para el taller:

- El archivo `compose.yaml` levanta dos servidores de base de datos libSQL, que es el mismo motor que uso en producción. Antes hacía las pruebas contra un archivo local y no era lo mismo: los errores de concurrencia y de claves repetidas solo aparecen contra el servidor de verdad.
- El archivo `.devcontainer/Dockerfile` fija la versión de Node en 22.12 y trae el navegador de las pruebas automatizadas. Esto salió de un problema real: si la terminal abre con Node 20, el comando de compilación falla. Dentro del contenedor eso ya no pasa.

La Tabla 3 muestra que los conceptos son los mismos, solo cambia la aplicación.

Tabla 3

Los mismos conceptos en la aplicación de la guía y en la mía

Concepto	En Dino Run	En mi proyecto
Imagen	sena-docker-flask-app-app	libsql-server y la imagen del entorno de desarrollo
Contenedor	dino-game	codebymike-libsql-main y codebymike-libsql-demo
Dockerfile	Python 3.12 con Flask	Node 22.12 con Chromium
Puerto publicado	5000:5000	127.0.0.1:8080:8080 y 127.0.0.1:8081:8080
Guardar los datos	bind mount <code>./data:/app/data</code>	un volumen de Docker, y en el ejercicio 5 lo cambio a bind mount
Variables de entorno	DATABASE_PATH y SECRET_KEY	SQLD_NODE y SQLD_DB_PATH
Compose	un servicio	dos servicios que comparten configuración

Nota. Elaboración propia.

Ejercicios

Sección 16 de la guía. Todos los comandos los corrí estando en la carpeta de mi proyecto.

Ejercicio 1. Comprobar que el Motor Sirve

Lo que pide: comprobar que el motor funciona y puede ejecutar imágenes.

Sin interfaz gráfica el ciclo es el mismo: bajar una imagen, crear el contenedor y correrlo.

```
docker run hello-world
docker images | head
docker ps -a | head
```

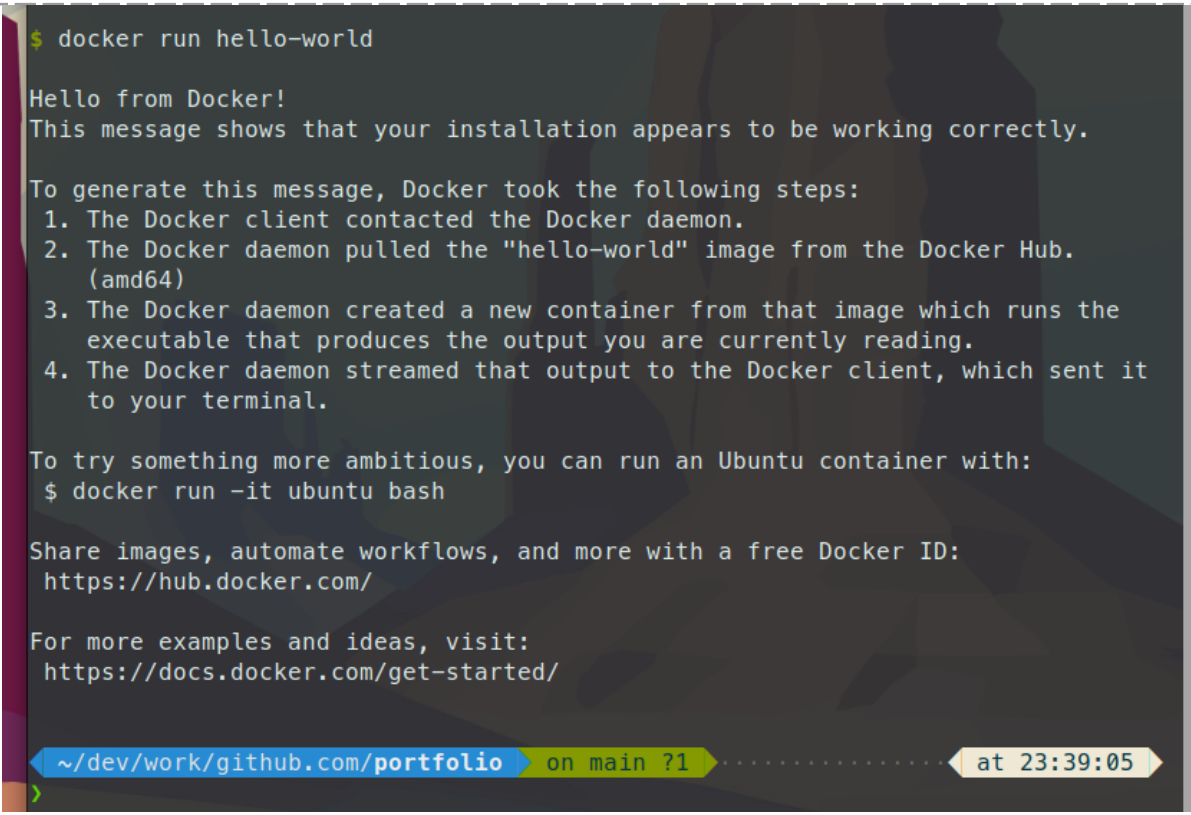
Algo que no tenía claro antes y me quedó claro con esto: docker run no es un solo paso. Busca la imagen en el disco, si no la encuentra la descarga, crea el contenedor y lo arranca. Por eso la guía usa hello-world como prueba, porque con un comando se ven las cuatro cosas.

El problema de hello-world es que se muere de una y no queda nada que mirar. Para que la evidencia quedara más completa levanté también un servidor web con un puerto publicado:

```
docker run -d --name prueba-sena -p 8085:80 nginx
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://localhost:8085
docker ps --filter name=prueba-sena
docker logs prueba-sena
docker rm -f prueba-sena
```

Figura 4

La imagen de prueba hello-world



```
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

~/dev/work/github.com/portfolio on main ?1 at 23:39:05

Donde: Terminal, en cualquier carpeta.

Comandos a ejecutar:

```
docker run hello-world
```

Debe verse: El mensaje Hello from Docker! con la explicación de los cuatro pasos. Si la imagen no estaba descargada también salen las líneas de descarga, y eso hace mejor la evidencia.

Nota. Captura tomada en Ubuntu 24.04.4 LTS con Docker Engine 29.6.2. Elaboración propia.

Figura 5*Un contenedor con puerto publicado*

```
$ docker run -d --name prueba-sena -p 8085:80 nginx
c8af5a409c4e211958e959b0ac377c7101c579e286d9a48f81458ad28be49d1b

$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://localhost:8085
HTTP 200

$ docker ps --filter name=prueba-sena --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
NAMES          STATUS          PORTS
prueba-sena    Up 2 seconds    0.0.0.0:8085->80/tcp, [::]:8085->80/tcp

(el contenedor queda arriba; para borrarlo: docker rm -f prueba-sena)
```

~/dev/work/github.com/portfolio on main ?1 at 23:39:34

Donde: Terminal, en cualquier carpeta.**Comandos a ejecutar:**

```
docker run -d --name prueba-sena -p 8085:80 nginx
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://localhost:8085
docker ps --filter name=prueba-sena
```

Debe verse: El identificador largo del contenedor, la respuesta HTTP 200 y la fila del contenedor con el puerto 0.0.0.0:8085->80/tcp. Es lo mismo que darle clic al enlace del puerto en la pantalla Containers.*Nota.* Captura tomada en Ubuntu 24.04.4 LTS con Docker Engine 29.6.2. Elaboración propia.

Ejercicio 2. Construir la Imagen y Revisarla

Lo que pide: construir la aplicación y entender el resultado desde Images y Builds.

Construí el Dockerfile del entorno de desarrollo de mi proyecto:

```
docker build -t codebymike-dev:sena .devcontainer/
```

Para revisarlo después usé estos comandos, que muestran lo mismo que las pantallas Builds e

Images:

```
# lo que en Docker Desktop es la pantalla Builds
docker buildx history ls
docker buildx history logs <BUILD_ID>
```



```
# lo que es la pantalla Images
docker images codebymike-dev

# las capas de la imagen, con el peso de cada una
docker history codebymike-dev:sena --format 'table {{.Size}}\
t{{.CreatedBy}}'
```

La parte que más me gustó de este ejercicio fue la caché. La guía explica en la sección 6.2 por qué se copia primero requirements.txt y después el código, y uno lo lee y dice bueno. Pero al correr el mismo build dos veces se ve solo: la segunda vez todos los pasos salen marcados como CACHED y termina en menos de un segundo.

```
docker build -t codebymike-dev:sena .devcontainer/ # ahora sale todo
CACHED
```

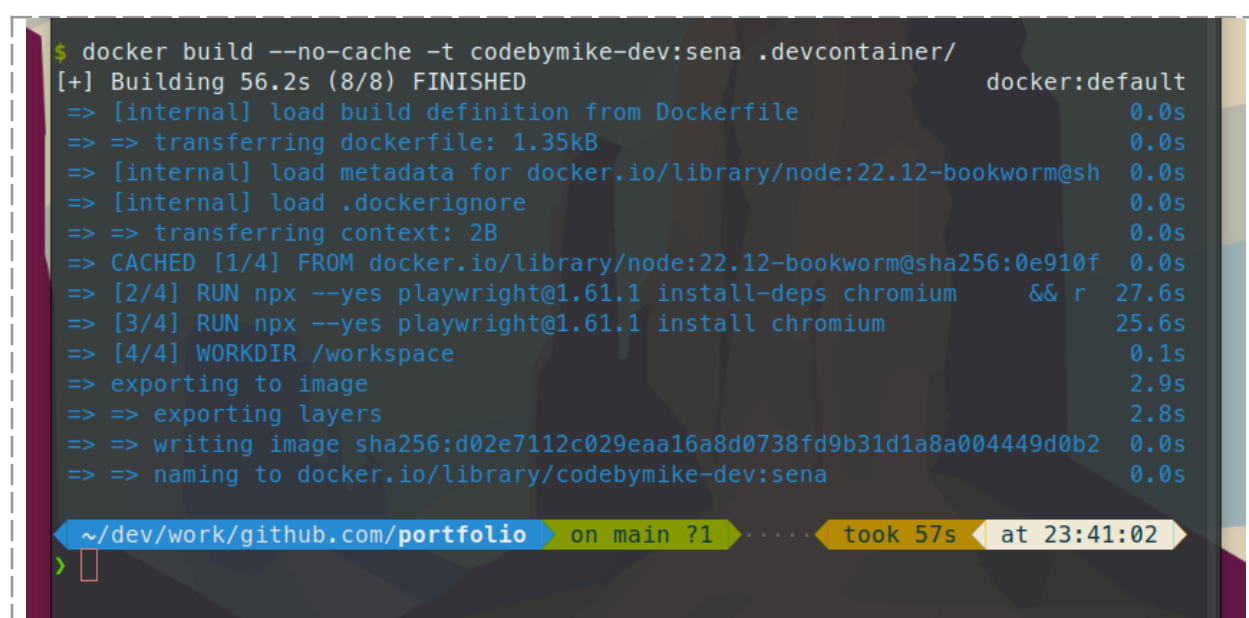
Comparando mi Dockerfile con el de Dino Run encontré que hacen casi lo mismo pero con otras herramientas. Los dos parten de una imagen oficial, instalan dependencias, copian lo que hace falta, bajan los permisos con USER y definen la carpeta de trabajo. Hay dos diferencias que sí vale la pena mencionar:

- Mi imagen base va fijada con un digest, que es una huella del contenido, en vez de una etiqueta como python:3.12-slim. Una etiqueta puede cambiar mañana y apuntar a otra imagen; el digest no cambia nunca. Es más incómodo de escribir pero es la única forma de que la construcción sea igual dentro de seis meses.
- Yo uso una instrucción ENV dentro del Dockerfile para la ruta de los navegadores, mientras que la guía prefiere poner las variables en compose.yaml. Las dos formas son válidas y la diferencia es cuándo cambia el valor: si es igual siempre, va en la imagen; si cambia según dónde se ejecute, va en Compose.

Del `.dockerignore` aprendí algo que la guía menciona de pasada. Uno cree que es para que el build sea más rápido, y sí, pero lo importante es otra cosa: si un archivo con contraseñas entra al contexto puede quedarse guardado en una capa de la imagen aunque el Dockerfile nunca lo copie. Por eso en el mío excluyo `.env`, la carpeta `.git`, `node_modules` y los documentos en PDF y Word que tengo dentro del repositorio.

Figura 6

Primera construcción de la imagen



```
$ docker build --no-cache -t codebymike-dev:sena .devcontainer/
[+] Building 56.2s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 1.35kB                            0.0s
=> [internal] load metadata for docker.io/library/node:22.12-bookworm@sh 0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                     0.0s
=> CACHED [1/4] FROM docker.io/library/node:22.12-bookworm@sha256:0e910f 0.0s
=> [2/4] RUN npx --yes playwright@1.61.1 install-deps chromium && r 27.6s
=> [3/4] RUN npx --yes playwright@1.61.1 install chromium      25.6s
=> [4/4] WORKDIR /workspace                                    0.1s
=> exporting to image                                           2.9s
=> => exporting layers                                           2.8s
=> => writing image sha256:d02e7112c029eaa16a8d0738fd9b31d1a8a004449d0b2 0.0s
=> => naming to docker.io/library/codebymike-dev:sena          0.0s

~/dev/work/github.com/portfolio on main ?1 ... took 57s at 23:41:02
> 
```

Donde: Terminal, en la carpeta del proyecto.

Comandos a ejecutar:

```
docker build -t codebymike-dev:sena .devcontainer/
```

Debe verse: Los pasos numerados, la descarga de la imagen base y el tiempo total al final. Equivale a la pantalla Builds con el estado Completed.

Nota. Captura tomada en Ubuntu 24.04.4 LTS con Docker Engine 29.6.2. Elaboración propia.

Figura 7

La segunda construcción usa la caché

```
$ docker build -t codebymike-dev:sena .devcontainer/
[+] Building 0.1s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 1.35kB                            0.0s
=> [internal] load metadata for docker.io/library/node:22.12-bookworm@sh 0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/4] FROM docker.io/library/node:22.12-bookworm@sha256:0e910f435308c 0.0s
=> CACHED [2/4] RUN npx --yes playwright@1.61.1 install-deps chromium 0.0s
=> CACHED [3/4] RUN npx --yes playwright@1.61.1 install chromium 0.0s
=> CACHED [4/4] WORKDIR /workspace                             0.0s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:d02e7112c029eaa16a8d0738fd9b31d1a8a004449d0b2 0.0s
=> => naming to docker.io/library/codebymike-dev:sena          0.0s

~/dev/work/github.com/portfolio on main ?1 ..... at 23:41:52
>
```

Donde: Terminal, justo después de la captura anterior.

Comandos a ejecutar:

```
docker build -t codebymike-dev:sena .devcontainer/
```

Debe verse: Todos los pasos con la palabra CACHED y un tiempo total de menos de un segundo. Es la prueba de que las capas se reutilizan.

Nota. Captura tomada en Ubuntu 24.04.4 LTS con Docker Engine 29.6.2. Elaboración propia.

Figura 8*Historial de construcciones, imagen y capas*

```
$ docker buildx history ls | head -5
```

BUILD ID	NAME	STATUS	CREATED AT	DURAT
ION				
9qi1qc8c6582qvanehandqc12	.devcontainer	Completed	13 seconds ago	0.1s
zdwe4w6fffomtvj7ctcb0jr8i	.devcontainer	Completed	2 minutes ago	56.2s
5eoryr98yl6hknetxwl4xx421	.devcontainer	Completed	12 days ago	50.4s
9bph36fytrnjdz3g5jyph1qq1	aws-lambda-pptx	Completed	4 months ago	1m 31s

```
$ docker images codebymike-dev
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
codebymike-dev:sena	d02e7112c029	2.21GB	0B	

```
$ docker history codebymike-dev:sena --format 'table {{.Size}}\t{{.CreatedBy}}'
```

SIZE	CREATED BY
0B	WORKDIR /workspace
0B	ENV PLAYWRIGHT_BROWSERS_PATH=/home/node/.cac...
739MB	RUN 1 PLAYWRIGHT_VERSION=1.61.1 /bin/sh -c ...
0B	USER node
357MB	RUN 1 PLAYWRIGHT_VERSION=1.61.1 /bin/sh -c ...
0B	ARG PLAYWRIGHT_VERSION=1.61.1
0B	CMD ["node"]
0B	ENTRYPOINT ["docker-entrypoint.sh"]
388B	COPY docker-entrypoint.sh /usr/local/bin/ # ...
5.34MB	RUN /bin/sh -c set -ex && export GNUPGHOME...
0B	ENV YARN_VERSION=1.22.22
184MB	RUN /bin/sh -c ARCH= && dpkgArch="\$(dpkg --p...
0B	ENV NODE_VERSION=22.12.0
8.94kB	RUN /bin/sh -c groupadd --gid 1000 node &&...
588MB	RUN /bin/sh -c set -ex; apt-get update; ap...
177MB	RUN /bin/sh -c set -eux; apt-get update; a...
47.7MB	RUN /bin/sh -c set -eux; apt-get update; a...
117MB	# debian.sh --arch 'amd64' out/ 'bookworm' '...

Donde: Terminal, en la carpeta del proyecto.

Comandos a ejecutar:

```
docker buildx history ls
docker images codebymike-dev
docker history codebymike-dev:sena --format 'table {{.Size}}\t{{.CreatedBy}}'
```

Debe verse: El registro de la construcción con su duración, la fila de la imagen con etiqueta y tamaño, y el desglose de capas donde se ve cuánto pesa cada instrucción.

Nota. Captura tomada en Ubuntu 24.04.4 LTS con Docker Engine 29.6.2. Elaboración propia.

Ejercicio 3. Levantar Todo con Docker Compose

Lo que pide: ejecutar la aplicación con Compose y entrar desde el navegador sin tener que configurar cada opción a mano.

En mi proyecto esto está metido en un script de npm, que viene siendo lo mismo que el `iniciar_juego.bat` de la guía pero para Linux:

```
npm run db:up          # levanta los contenedores y espera a que respondan
docker compose ps
```

El comando que corre por debajo es este:

```
docker compose up -d
docker compose ps --format 'table {{.Name}}\t{{.Status}}\t{{.Ports}}'
```

Y para comprobar por el puerto, que es el equivalente de darle clic a 5000:5000:

```
curl -s -o /dev/null -w 'principal HTTP %{http_code}\n'
http://127.0.0.1:8080/health
curl -s -o /dev/null -w 'demo HTTP %{http_code}\n'
http://127.0.0.1:8081/health
```

Después levanto la aplicación apuntando a esas bases:

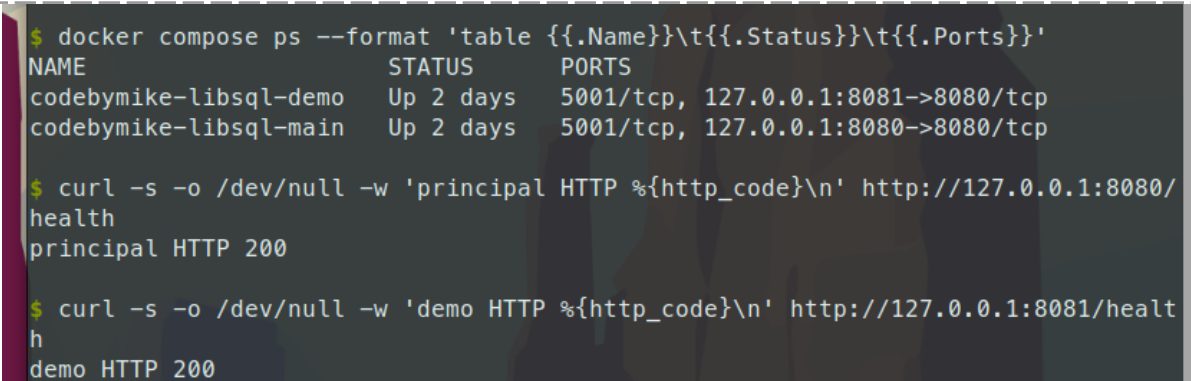
```
npm run db:seed
npm run dev:carga      # abre en http://localhost:4400
```

Mi `compose.yaml` tiene tres cosas que el de la guía no, y las entendí mejor haciendo esta comparación. La primera es que los dos servicios comparten un bloque de configuración escrito una sola vez, usando lo que en YAML se llama un ancla; como la guía tiene un solo servicio, nunca se topa con ese problema. La segunda es que mis puertos están escritos como `127.0.0.1:8080:8080` y no como `8080:8080`. La diferencia es grande: la forma corta, en Linux,

deja el servicio abierto a toda la red local, mientras que la mía solo lo deja disponible en mi propio computador. La tercera es que el contenedor arranca con casi todos los permisos del núcleo quitados y solo con cuatro habilitados, que fueron los que necesitó la base de datos para poder iniciar.

Figura 9

Los servicios levantados y respondiendo



```
$ docker compose ps --format 'table {{.Name}}\t{{.Status}}\t{{.Ports}}'
NAME                                STATUS      PORTS
codebymike-libsql-demo             Up 2 days   5001/tcp, 127.0.0.1:8081->8080/tcp
codebymike-libsql-main             Up 2 days   5001/tcp, 127.0.0.1:8080->8080/tcp

$ curl -s -o /dev/null -w 'principal HTTP %{http_code}\n' http://127.0.0.1:8080/health
principal HTTP 200

$ curl -s -o /dev/null -w 'demo HTTP %{http_code}\n' http://127.0.0.1:8081/health
demo HTTP 200
```

Donde: Terminal, en la carpeta del proyecto.

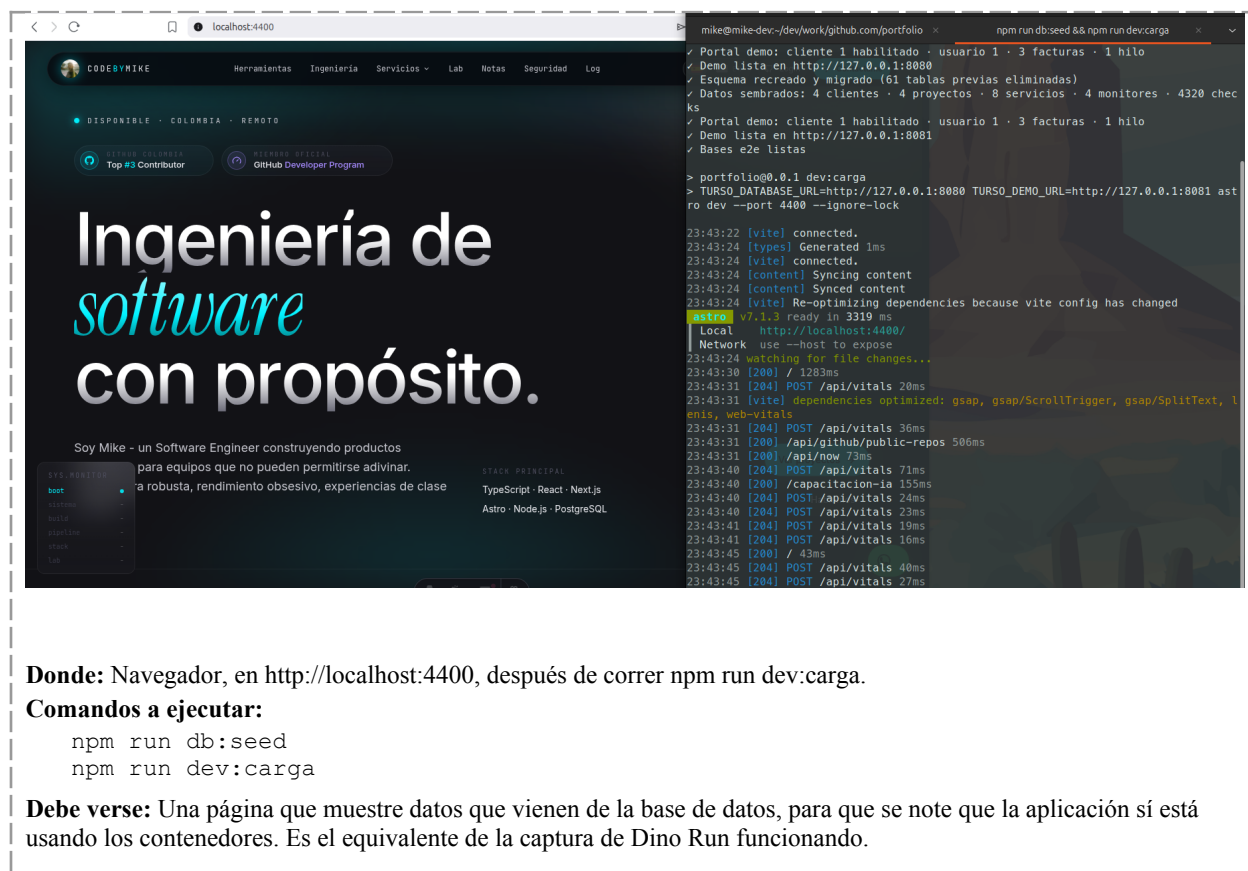
Comandos a ejecutar:

```
docker compose ps --format 'table {{.Name}}\t{{.Status}}\t{{.Ports}}'
curl -s -o /dev/null -w 'principal HTTP %{http_code}\n' http://127.0.0.1:8080/health
curl -s -o /dev/null -w 'demo HTTP %{http_code}\n' http://127.0.0.1:8081/health
```

Debe verse: Los dos contenedores en estado Up con sus puertos, y debajo las dos respuestas HTTP 200. Es la pantalla Containers con el grupo de Compose desplegado.

Figura 10

La aplicación abierta en el navegador



Ejercicio 4. Las Variables de Entorno y la Clave Secreta

Lo que pide: separar las variables normales de las secretas y comprobar que la variable existe sin mostrar su valor.

Las variables que reciben mis contenedores se ven así:

```
docker inspect codebymike-libsqli-main \
  --format '{{range .Config.Env}}{{println .}}{{end}}'

# SQLD_NODE=primary
# SQLD_DB_PATH=/var/lib/sqlld/main.db
```

SQLD_DB_PATH es lo mismo que DATABASE_PATH en Dino Run: le dice al programa dónde escribir. Lo que hay que cuidar es que esa ruta coincida con la del montaje. Si uno cambia una y olvida la otra, el programa escribe en un lugar que no se está guardando y los datos se pierden apenas se recrea el contenedor.

Del lado de mi aplicación las variables se dividen en tres grupos, que es justo lo que pide el ejercicio. Las normales, como la dirección de la base de datos local, se pueden mostrar sin problema. Las opcionales, como las de notificaciones, se pueden dejar vacías y la aplicación sigue funcionando, solo que no envía nada. Y las secretas, como la clave de cifrado, viven en un archivo .env que está excluido tanto del repositorio como del contexto de construcción.

Para la evidencia hay que comprobar que la variable existe sin dejar ver el valor. La forma que encontré es cortar todo lo que va después del signo igual:

```
# muestra solo los nombres, sin los valores
docker inspect codebymike-libsqli-main \
  --format '{{range .Config.Env}}{{println .}}{{end}}' | cut -d= -f1

# confirma que la clave existe, sin imprimirla
grep -c '^ENCRYPTION_KEY=' .env          # -> 1
```


Figura 11*Variables de entorno sin mostrar los valores secretos*

```

$ docker inspect codebymike-libsql-main --format '{{range .Config.Env}}{{println .}}
{{end}}'
SQLD_NODE=primary
SQLD_DB_PATH=/var/lib/sqlite/main.db
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

$ docker inspect codebymike-libsql-main --format '{{range .Config.Env}}{{println .}}
{{end}}' | cut -d= -f1
SQLD_NODE
SQLD_DB_PATH
PATH

$ grep -c '^ENCRYPTION_KEY=' .env
1

```

Donde: Terminal, en la carpeta del proyecto.

Comandos a ejecutar:

```

docker inspect codebymike-libsql-main \
  --format '{{range .Config.Env}}{{println .}}{{end}}'

docker inspect codebymike-libsql-main \
  --format '{{range .Config.Env}}{{println .}}{{end}}' | cut -d= -f1

grep -c '^ENCRYPTION_KEY=' .env

```

Debe verse: Primero las variables del contenedor con sus valores, que no son secretos; después la misma consulta cortada a solo los nombres; y por último el conteo que prueba que la clave existe sin mostrarla. Antes de tomar la captura hay que revisar que no haya quedado ninguna clave visible más arriba en la terminal.

Ejercicio 5. Que los Datos No Se Pierdan

Lo que pide: guardar los datos con el bind mount de compose.yaml, reiniciar el contenedor y comprobar que siguen ahí.

Mi proyecto no usa bind mount sino un volumen de Docker, que para bases de datos funciona mejor porque Docker se encarga de los permisos y los archivos no quedan regados en la carpeta

del proyecto. Para el ejercicio armé un archivo aparte que cambia solo esa parte, sin dañar la configuración normal:

```
services:
  libsql-main:
    volumes:
      - ../data/libsql-main:/var/lib/sqlite
```

Y lo levanté combinando los dos archivos:

```
# antes: montaje tipo volume
docker inspect codebymike-libsql-main \
  --format '{{range .Mounts}}{{.Type}} {{.Source}}{{println}}{{end}}'

docker compose -f compose.yaml \
  -f docs/manuales-sena/compose.sena-bind.yaml up -d

# después: montaje tipo bind, apuntando a mi carpeta
docker inspect codebymike-libsql-main \
  --format '{{range .Mounts}}{{.Type}} {{.Source}}{{println}}{{end}}'

ls -la .data/libsql-main
```

La prueba de que los datos aguantan es la misma que propone la guía con el récord del juego: guardar algo, reiniciar y volver a mirar. Para no escribir la consulta a mano cada vez armé un script pequeño que inserta una fila y otro modo que la lee.

```
node scripts/prueba-persistencia.mjs escribir
docker compose restart libsql-main
node scripts/prueba-persistencia.mjs leer
du -sh .data/libsql-main
```

Antes del reinicio la tabla tenía una fila y después seguía teniendo la misma, con el texto y la fecha intactos.

Para volver a como estaba antes:

```
docker compose up -d --force-recreate
```

Aquí me pasó el único problema que no esperaba. Cuando quise borrar la carpeta .data para dejar todo limpio, me salió Permission denied, y eso que era una carpeta dentro de mi propio proyecto. Lo que ocurre es que los archivos los creó el usuario que corre adentro del contenedor, que tiene otro número de usuario, y desde afuera yo no soy dueño de esos archivos. La solución fue borrarlos desde un contenedor:

```
docker run --rm -v "$PWD/.data:/x" alpine rm -rf /x/libsql-main
```

Estuve leyendo y esto en Windows no pasa, porque el acceso a los archivos va por una capa intermedia que empareja los permisos. Me sirvió para entender por qué en Linux se prefieren los volúmenes de Docker para las bases de datos y los bind mounts se dejan más que todo para el código mientras uno programa.

Figura 12*El montaje cambia de volumen a bind mount*

```

$ docker inspect codebymike-libsql-main --format '{{range .Mounts}}{{.Type}} {{.Source}}{{println}}{{end}}'
volume /var/lib/docker/volumes/codebymike_libsql-main-data/_data

$ docker compose -f compose.yaml -f docs/manuales-sena/compose.sena-bind.yaml up -d
[+] up 2/2
✓ Container codebymike-libsql-demo Running 0.0s
✓ Container codebymike-libsql-main Started 0.6s

$ docker inspect codebymike-libsql-main --format '{{range .Mounts}}{{.Type}} {{.Source}}{{println}}{{end}}'
bind /home/mike/dev/work/github.com/portfolio/.data/libsql-main

$ ls -la .data/libsql-main
total 12
drwxr-xr-x 3 root root 4096 Aug 12 23:44 .
drwxr-xr-x 3 root root 4096 Aug 12 23:44 ..
drwxr-xr-x 4 666 666 4096 Aug 12 23:44 main.db

```

Donde: Terminal, en la carpeta del proyecto.**Comandos a ejecutar:**

```

docker inspect codebymike-libsql-main \
  --format '{{range .Mounts}}{{.Type}} {{.Source}}{{println}}{{end}}'

docker compose -f compose.yaml \
  -f docs/manuales-sena/compose.sena-bind.yaml up -d

docker inspect codebymike-libsql-main \
  --format '{{range .Mounts}}{{.Type}} {{.Source}}{{println}}{{end}}'

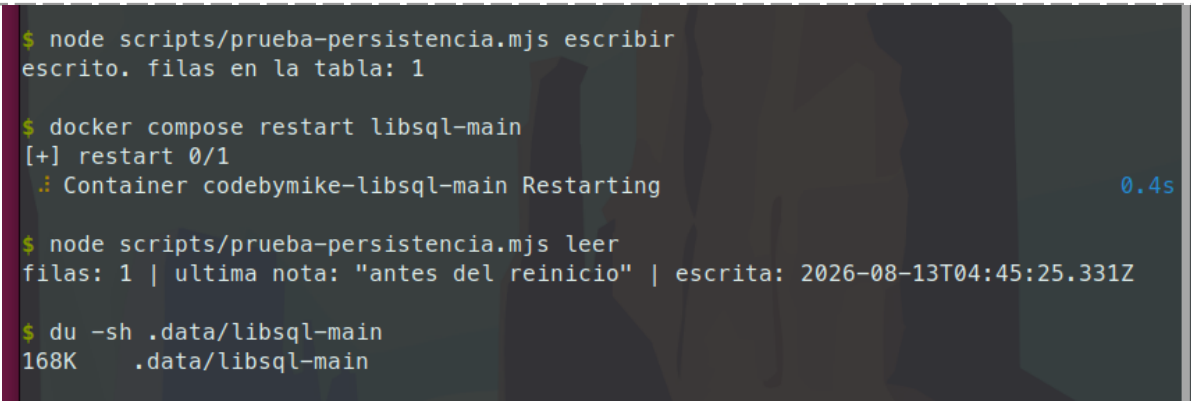
ls -la .data/libsql-main

```

Debe verse: Las dos consultas una debajo de la otra: la primera dice volume y una ruta larga de Docker, la segunda dice bind y la ruta de mi proyecto. Al final el listado de la carpeta que apareció. Es la pestaña Bind mounts de Docker Desktop.

Figura 13

Los datos siguen ahí después de reiniciar



```

$ node scripts/prueba-persistencia.mjs escribir
escrito. filas en la tabla: 1

$ docker compose restart libsql-main
[+] restart 0/1
.: Container codebymike-libsql-main Restarting 0.4s

$ node scripts/prueba-persistencia.mjs leer
filas: 1 | ultima nota: "antes del reinicio" | escrita: 2026-08-13T04:45:25.331Z

$ du -sh .data/libsql-main
168K    .data/libsql-main

```

Donde: Terminal, en la carpeta del proyecto.

Comandos a ejecutar:

```

node scripts/prueba-persistencia.mjs escribir
docker compose restart libsql-main
node scripts/prueba-persistencia.mjs leer
du -sh .data/libsql-main

```

Debe verse: La consulta antes del reinicio, el reinicio del contenedor y la consulta después, con el mismo dato. Esta es la captura que corresponde al récord que se conserva en el ejemplo de la guía.

Ejercicio 6. El Puerto Ocupado

Lo que pide: diagnosticar un puerto ocupado y cambiar el puerto del computador en `compose.yaml`.

Este error no toca esperarlo, se puede provocar. Como mi base de datos ya tiene tomado el 8080, pedí ese mismo puerto para otro contenedor:

```
docker run -d --name choque -p 127.0.0.1:8080:80 nginx
```

Y salió el mensaje de la guía:

```

Error response from daemon: failed to set up container networking:
driver failed programming external connectivity on endpoint choque:
Bind for 127.0.0.1:8080 failed: port is already allocated

```

Para saber quién lo tiene ocupado hay dos comandos, y me parece importante mostrar los dos:

```
# si el que lo ocupa es un contenedor
docker ps --filter publish=8080 --format '{{.Names}} -> {{.Ports}}'

# si el que lo ocupa es un programa del computador
ss -ltnp | grep :8080
```

El segundo es una ventaja de estar en Linux. Cuando el puerto lo tiene un programa instalado en el equipo y no un contenedor, Docker Desktop no lo puede ver y uno se queda sin saber qué pasa.

La solución es la que dice la guía, cambiar el puerto del lado izquierdo:

```
docker rm -f choque
docker run -d --name choque -p 127.0.0.1:8086:80 nginx
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8086
docker rm -f choque
```

Y en Compose sería cambiar 8080:8080 por 8085:8080, igual que la guía cambia 5000:5000 por 5001:5000. El número de la derecha lo manda la aplicación, uno no lo puede escoger, y el de la izquierda es el que uno elige. Por eso siempre se mueve el izquierdo, y por eso EXPOSE no publica nada: solo deja anotado cuál es el puerto de adentro.

Figura 14*Puerto ocupado, diagnóstico y solución*

```

$ docker run -d --name choque -p 127.0.0.1:8080:80 nginx
8b5a566727094f6a1cd1235b823505be18b10ef06a56d66edd741585c770e913
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint choque (2853597221d370d0408428645b104763b9933076e47f61e975dd9f909a27317d): Bind for 127.0.0.1:8080 failed: port is already allocated

Run 'docker run --help' for more information

$ docker ps --filter publish=8080 --format '{{.Names}} -> {{.Ports}}'
codebymike-libsqli-main -> 5001/tcp, 127.0.0.1:8080->8080/tcp

$ ss -ltnp | grep :8080
LISTEN 0      4096      127.0.0.1:8080      0.0.0.0:*

$ docker rm -f choque
choque

$ docker run -d --name choque -p 127.0.0.1:8086:80 nginx
031c772856f163ca81c14e50c2c1df95ebe6e751f316cb7aa574ceee3a0ead19

$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8086
HTTP 200

$ docker rm -f choque
choque

```

Donde: Terminal, en la carpeta del proyecto.**Comandos a ejecutar:**

```

docker run -d --name choque -p 127.0.0.1:8080:80 nginx
docker ps --filter publish=8080 --format '{{.Names}} -> {{.Ports}}'
ss -ltnp | grep :8080
docker rm -f choque
docker run -d --name choque -p 127.0.0.1:8086:80 nginx
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8086
docker rm -f choque

```

Debe verse: Todo seguido: el mensaje port is already allocated, el contenedor que tenía ocupado el puerto, la comprobación a nivel del sistema y el mismo contenedor funcionando en el otro puerto con respuesta HTTP 200.

Preguntas de la Sección 17

Qué Significa Empaquetar una Aplicación

Empaquetar es encapsular el programa con todo lo que este necesita para funcionar: el lenguaje, las librerías, la configuración y el comando con el que arranca. Todo en una sola pieza que no depende de cómo esté configurado el computador donde va a correr.

El caso de que algo funcione en un equipo y falle en otro, en mi caso la terminal a veces abre con Node 20 y el proyecto necesita 22.12. Mismo repositorio, mismo comando, y en un computador compila y en otro no. Antes lo resolvía cambiando la versión a mano cada vez, hasta que arme el archivo del entorno de desarrollo.

Diferencia entre Imagen, Contenedor y Dockerfile

El Dockerfile es como una receta: un archivo de texto con los pasos y la imagen es el plato ya preparado, que no cambia y se puede guardar y el contenedor es cuando uno se sienta a comerlo, o sea la ejecución.

De una misma imagen pueden salir varios contenedores. En mi proyecto se ve directo: los dos contenedores de base de datos usan exactamente la misma imagen, pero uno tiene los datos reales de desarrollo y el otro los de la demostración, y cada uno escucha en un puerto distinto.

Sobre las Pantallas y las Capas

Las capas y la caché las trabajé en el ejercicio 2, el bind mount en el ejercicio 5 y Compose en el ejercicio 3. La actividad que propone cambiar algo y volver a construir la hice cambiando la versión del navegador en el Dockerfile: al reconstruir se ve que los pasos de antes de esa línea

siguen saliendo como CACHED y los de después se vuelven a ejecutar. La imagen base no se descarga otra vez.

Conclusiones

Los temas de la guía los trabajé todos: imagen, contenedor, Dockerfile, contexto de construcción, puertos, variables de entorno, montajes, persistencia, logs y Compose. Lo que cambió fue que en vez de darle clic a una pantalla tuve que escribir el comando.

Manejar el motor directo me mostró cosas que la interfaz gráfica esconde y que creo que vale la pena saber: que Docker es un servicio del sistema y no un programa que uno abre, que estar en el grupo docker es casi ser administrador, que los archivos de un bind mount quedan con el usuario del contenedor, y que un puerto ocupado se puede diagnosticar a nivel de todo el sistema y no solo entre contenedores.

También me sirvió comparar el ejemplo de la guía con mi proyecto, porque me obligó a explicar decisiones que ya estaban tomadas y que nunca había puesto por escrito, como ejemplo, por qué mi aplicación usa Docker para desarrollar pero no para publicarse. Esa última fue la que más me costó justificar y creo que es la que más aprendí, porque saber dónde no usar una herramienta también es parte de saber usarla.

Referencias

Docker Inc. (2026a). Docker overview. Docker Docs.

<https://docs.docker.com/get-started/docker-overview/>

Docker Inc. (2026b). Docker Desktop WSL 2 backend on Windows. Docker Docs.

<https://docs.docker.com/desktop/features/wsl/>

Docker Inc. (2026c). Install Docker Engine on Ubuntu. Docker Docs.

<https://docs.docker.com/engine/install/ubuntu/>

Servicio Nacional de Aprendizaje. (2026). Instalación y uso de Docker [Guía de aprendizaje].

Formación en Ambientes Virtuales de Aprendizaje.